Computer Architecture Laboratory

Milestone 2: Single-cycle Processor

Status: Version 1.0
Author: Khai Pham
Vy Luong
Date: 2026-Oct-13

1. Overview

In Milestone 2, students design and verify a single-cycle RV32I processor based on the architecture covered in lectures. The processor must execute the required instructions (Table 3) and supplied programs in simulation.

A project template and verification environment are provided. Students must preserve the specified module names and interfaces to ensure compatibility with the testbench and grading environment.

2. Objectives

- Review understanding of SystemVerilog
- Review understanding of RV32I instructions
- Design a single-cycle RV32I processor
- Execute simple software programs in simulation.

3. Requirements

- Complete all required submissions and assessment activities.
- Submit all work before the deadline unless an extension is approved.
- Submit only original work. Copying from other students or groups is prohibited.
- FPGA implementation is not required for this milestone.
- Follow the prescribed submission procedure. Incorrect submissions may not be assessed.

> ! **WARNING Restricted SystemVerilog Constructs**
>
> Unless explicitly permitted, do not use addition (`+`), subtraction (`-`), comparison (`<`, `>`, `<=`, `>=`), shift (`<<`, `>>`, `>>>`), multiplication (`*`), division (`/`), or modulo (`%`) operators in your design RTL.
>
> The use of `function` and `task` is also prohibited. Implement these operations using your own hardware logic or modules.
>
> These restrictions do not apply to testbenches or reference models used only for verification.

4. Specification: Single-Cycle Processor

The processor must use a **single-cycle architecture**, completing each instruction within one clock cycle, including fetch, decode, execution, memory access, and writeback.

The required execution performance is **Instructions per Cycle (IPC) = 1**, or approximately 1 (e.g. 0.99).

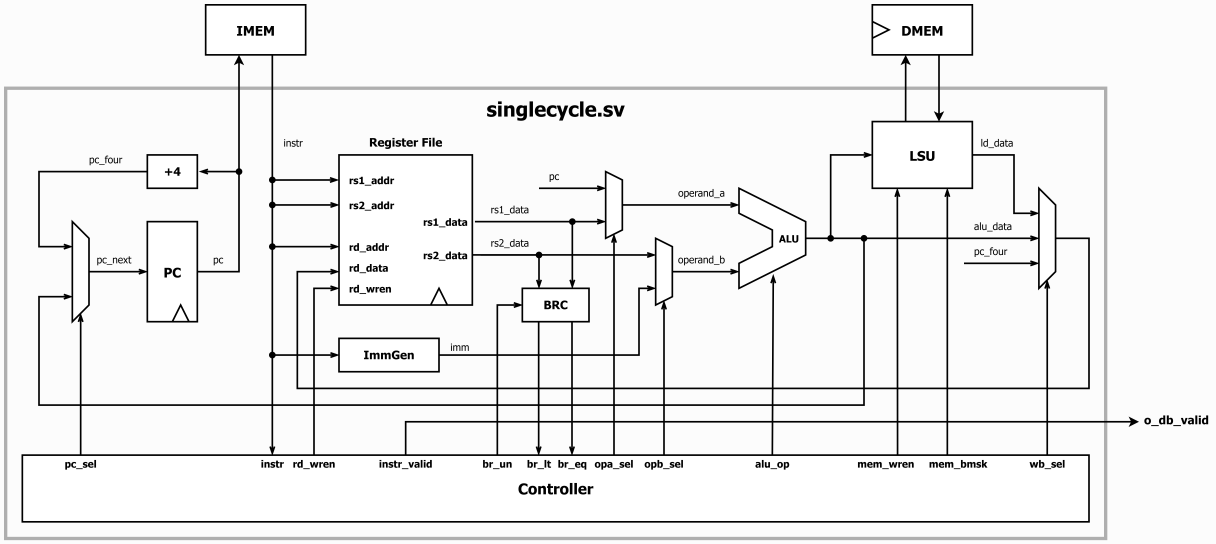


Figure 1: Single-cycle processor overview.

The architecture shown in Figure 1 is a recommended reference design, not a required internal structure. Students may organize the processor differently, provided that the design satisfies all functional requirements, preserves the specified input/output interfaces, and follows the procedures defined in this guideline.

It is recommended to implement and verify each functional unit independently before integrating the datapath and controller. The following sections specify their behavior and interfaces.

> **INFORMATION Required Instruction Set**
>
> Support is required only for the instructions listed in Table 3.
>
> Instructions such as **`FENCE`**, **`ECALL`**, **`EBREAK`**, and **`PAUSE`** are outside the scope of this milestone and do not need to be implemented.

4.1. Arithmetic Logic Unit (ALU) and Register File (Regfile)

Reuse the ALU and Register File from `Exercise 4` and `Exercise 5` of Milestone 1:

- The ALU performs arithmetic, logic, and address calculations.
- The Regfile supports two asynchronous read ports and one synchronous write port, with `x0` always reading as zero.

4.2. Branch Comparison Unit (BRC)

The Branch Comparison Unit (BRC) compares the two source-register values and provides the equality and less-than results needed by RV32I branches.

Table 1: Branch conditions of the RV32I instruction set.

Branch	Condition	Description
BEQ	`rs1 == rs2`	Taken when the two operands are equal.
BNE	`rs1 != rs2`	Taken when the two operands are different.
BLT	`rs1 < rs2`	Taken when `rs1` is less than `rs2` using signed comparison.
BGE	`rs1 >= rs2`	Taken when `rs1` is greater than or equal to `rs2` using signed comparison.
BLTU	`rs1 < rs2`	Taken when `rs1` is less than `rs2` using unsigned comparison.
BGEU	`rs1 >= rs2`	Taken when `rs1` is greater than or equal to `rs2` using unsigned comparison.

The controller combines these outputs with the decoded branch type to determine whether the branch is taken and to select the next program counter.

Table 2: Input and output signals of the Branch Comparison Unit.

Signal name	Width	Direction	Description
`i_rs1_data`	32	input	First source operand.
`i_rs2_data`	32	input	Second source operand.
`i_br_unsigned`	1	input	Selects unsigned comparison when high and signed comparison when low.
`o_br_equal`	1	output	High when `i_rs1_data == i_rs2_data`.
`o_br_less`	1	output	High when `i_rs1_data < i_rs2_data` under the selected comparison type.

4.3. Immediate Generator (ImmGen)

The Immediate Generator extracts and sign-extends immediate values from a 32-bit instruction according to the **U-**, **J-**, **I-**, **B-**, or **S-type** format selected by **i_imm_sel**. For B- and J-type immediates, bit 0 of the 32-bit output must be zero.

Table 3: RV32I instruction encodings grouped by instruction format.

31:25	24:20	19:15	14:12	11:7	6:0	Instruction
U-type						
imm[31:12]				rd	0110111	LUI
imm[31:12]				rd	0010111	AUIPC
J-type						
imm[20|10:1|11|19:12]				rd	1101111	JAL
I-type						
imm[11:0]		rs1	000	rd	1100111	JALR
imm[11:0]		rs1	000	rd	0000011	LB
imm[11:0]		rs1	001	rd	0000011	LH
imm[11:0]		rs1	010	rd	0000011	LW
imm[11:0]		rs1	100	rd	0000011	LBU
imm[11:0]		rs1	101	rd	0000011	LHU
imm[11:0]		rs1	000	rd	0010011	ADDI
imm[11:0]		rs1	010	rd	0010011	SLTI
imm[11:0]		rs1	011	rd	0010011	SLTIU
imm[11:0]		rs1	100	rd	0010011	XORI
imm[11:0]		rs1	110	rd	0010011	ORI
imm[11:0]		rs1	111	rd	0010011	ANDI
I-type shift						
0000000	shamt	rs1	001	rd	0010011	SLLI
0000000	shamt	rs1	101	rd	0010011	SRLI
0100000	shamt	rs1	101	rd	0010011	SRAI
B-type						
imm[12|10:5]	rs2	rs1	000	imm[4:1|11]	1100011	BEQ
imm[12|10:5]	rs2	rs1	001	imm[4:1|11]	1100011	BNE
imm[12|10:5]	rs2	rs1	100	imm[4:1|11]	1100011	BLT
imm[12|10:5]	rs2	rs1	101	imm[4:1|11]	1100011	BGE
imm[12|10:5]	rs2	rs1	110	imm[4:1|11]	1100011	BLTU
imm[12|10:5]	rs2	rs1	111	imm[4:1|11]	1100011	BGEU
S-type						
imm[11:5]	rs2	rs1	000	imm[4:0]	0100011	SB
imm[11:5]	rs2	rs1	001	imm[4:0]	0100011	SH
imm[11:5]	rs2	rs1	010	imm[4:0]	0100011	SW
R-type						
0000000	rs2	rs1	000	rd	0110011	ADD
0100000	rs2	rs1	000	rd	0110011	SUB
0000000	rs2	rs1	001	rd	0110011	SLL
0000000	rs2	rs1	010	rd	0110011	SLT
0000000	rs2	rs1	011	rd	0110011	SLTU
0000000	rs2	rs1	100	rd	0110011	XOR
0000000	rs2	rs1	101	rd	0110011	SRL
0100000	rs2	rs1	101	rd	0110011	SRA
0000000	rs2	rs1	110	rd	0110011	OR
0000000	rs2	rs1	111	rd	0110011	AND

Table 4: Input and output signals of the Immediate Generator.

Signal name	Width	Direction	Description
i_instr	32	input	Instruction containing the immediate fields.
i_imm_sel	3	input	Selects the immediate format.
o_imm	32	output	Generated 32-bit immediate value.

4.4. Instruction Memory (IMEM)

IMEM is a byte-addressed, little-endian memory with an asynchronous read port. It returns the 32-bit instruction selected by the program counter.

Use the IMEM template provided with the TA testbench. Keep its interface unchanged and set the parameters according to Table 10, so the given top-level **milestone2.sv** adapts easily to different memory maps and IMEM sizes.

{/} systemverilog

```systemverilog
module imem #(
    parameter logic [31:0] START_ADDR  = 32'h8000_0000,
    parameter int unsigned ADDR_WIDTH  = 19,
    parameter string       PROGRAMFILE = ""
) (
    input  logic        i_clk,
    input  logic [31:0] i_addr,
    output logic [31:0] o_data
);

// Insert your code here.

endmodule
```

Table 5: Parameters of the Instruction Memory.

Parameter	Type	Description
START_ADDR	32-bit value	Base address of the instruction memory.
ADDR_WIDTH	integer	Number of address bits. The memory contains $2^{\text{ADDR_WIDTH}}$ bytes.
PROGRAMFILE	string	Relative path of the program file used to initialize IMEM in simulation.

Initialize IMEM from **PROGRAMFILE** before simulation, for example with **$readmemh**:

{/} systemverilog

```systemverilog
initial begin
    $readmemh(PROGRAMFILE, imem);
end
```

> ! **WARNING PC reset value must match IMEM's START_ADDR**
>
> **START_ADDR** marks the start of the test program, and the first instruction sits at this address. Set the PC reset value to **START_ADDR** so the program is fetched correctly.

4.5. Data Memory (DMEM)

DMEM is a byte-addressed, little-endian memory used by load and store instructions. Reads are asynchronous; byte-enabled writes occur synchronously on the rising clock edge.

Use the DMEM template provided with the TA testbench. Keep its interface unchanged and set **START_ADDR** and **ADDR_WIDTH** according to Table 10.

{/} systemverilog

```systemverilog
module dmem #(
    parameter logic [31:0] START_ADDR = 32'h8008_0000,
    parameter int unsigned ADDR_WIDTH = 16
) (
    input  logic        i_clk,
    input  logic [31:0] i_addr,
    input  logic [31:0] i_wdata,
    input  logic [ 3:0] i_bmsk,
    input  logic        i_wren,
    output logic [31:0] o_rdata
);

// Insert your code here.

endmodule
```

Table 6: Parameters of the Data Memory.

Parameter	Type	Description
START_ADDR	32-bit value	Base address of the data memory.
ADDR_WIDTH	integer	Number of address bits. The memory contains $2^{\text{ADDR_WIDTH}}$ bytes.

Table 7: Ports of the Data Memory.

Port	Direction	Width	Description
i_clk	Input	1	Clock. Writes occur on the rising edge.
i_addr	Input	32	Effective address of the access.
i_wdata	Input	32	Data to be written.
i_bmsk	Input	4	Byte-enable mask. Bit n enables writing byte n of i_wdata.
i_wren	Input	1	Write enable. 1 for a write, 0 for a read.
o_rdata	Output	32	Data read from address i_addr.

4.6. Load/Store Unit (LSU)

The LSU connects the datapath to DMEM. It generates the memory request for every load and store and formats loaded data for register writeback.

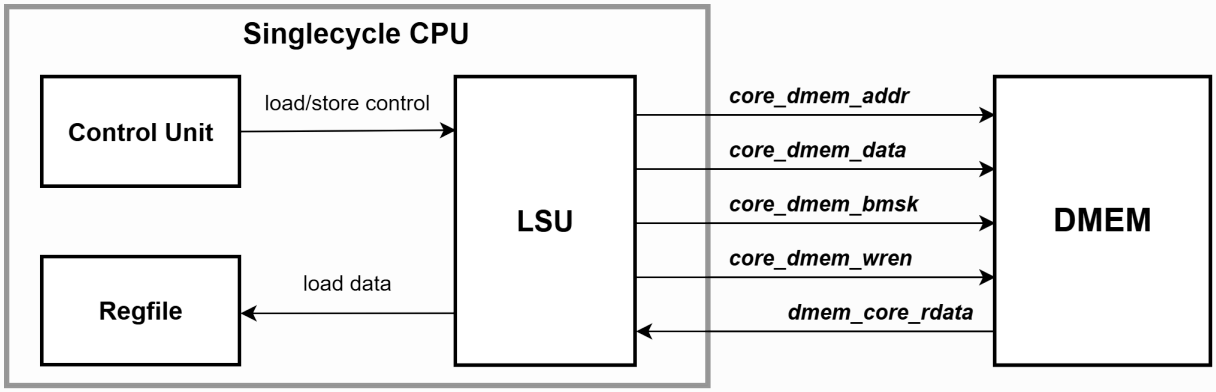


Figure 2: LSU sends requests to DMEM and processes the returned load data.

The LSU input interface is not specified. Students are responsible for designing it, including the control signals received from the controller.

Table 8: Output signals of the Load/Store Unit.

Signal name	Width	Direction	Description
`o_ld_data`	32	output	Processed load data for write-back.
`o_dmem_addr`	32	output	Address sent to DMEM.
`o_dmem_wdata`	32	output	Store data sent to DMEM.
`o_dmem_bmsk`	4	output	Byte mask sent to DMEM.
`o_dmem_wren`	1	output	Write enable sent to DMEM.

For stores (**`SB`**, **`SH`**, **`SW`**), the LSU aligns the store data and generates the byte mask **`o_dmem_bmsk`**.

For loads, it selects the addressed byte, halfword, or word from **`i_dmem_rdata`**. **`LB`** and **`LH`** sign-extend the result, **`LBU`** and **`LHU`** zero-extend it, and **`LW`** returns the full word. The result goes to the register file through **`o_ld_data`**.

4.7. Controller

The controller decodes the 32-bit instruction from IMEM and generates the datapath control signals. Its design is not specified; students are responsible for designing it.

It must validate instruction encodings and assert **`o_db_valid`** when a valid RV32I instruction completes. The signal must stay low during reset and for illegal instructions, which must not modify architectural state. The testbench samples **`o_db_valid`** on each rising clock edge to count executed instructions and measure IPC.

4.8. Single-Cycle Top Level

Students may structure the datapath of the single-cycle processor differently from the recommended structure in Figure 1. However, the input/output interface must strictly follow the specification below:

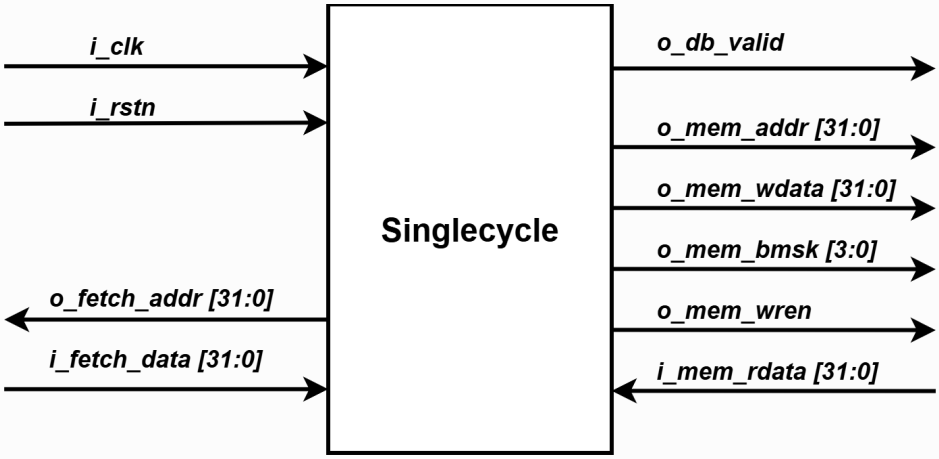


Figure 3: Single-cycle processor top-level interface.

Table 9: Port list of the `singlecycle` module, grouped by interface.

Signal name	Width	Direction	Description
Global Signals			
`i_clk`	1	input	Global clock signal.
`i_rstn`	1	input	Active-low reset signal.
Instruction Fetch Interface			
`o_imem_addr`	32	output	Byte address of the instruction to fetch.
`i_imem_rdata`	32	input	Instruction word returned from IMEM.
Data Memory Interface			
`o_dmem_addr`	32	output	Byte address of the data memory access.
`o_dmem_wdata`	32	output	Data to be written to DMEM.
`o_dmem_bmsk`	4	output	Byte mask selecting the active bytes of the access.
`o_dmem_wren`	1	output	Write enable, asserted for a store.
`i_dmem_rdata`	32	input	Data returned from DMEM for a load.
Debug Interface			
`o_db_valid`	1	output	Asserted when a valid instruction completes.

The module name `singlecycle` and its ports must exactly match the instantiation in the top-level module `milestone2.sv` of the provided testbench template.

5. Testbench

The lab TA provides a project template that includes the TA testbench and the test programs used to grade the final work.

Students are encouraged to write their own testbenches and tests to verify their design before running the provided testbench for the overall assessment.

5.1. Project Template

The template has the following structure:

{/} text

```
~/milestone2_template/
├── 00_src/
│   ├── milestone2.sv
│   ├── singlecycle.sv
│   ├── imem.sv
│   └── dmem.sv
├── 01_bench/
│   ├── bench_defines.svh
│   ├── tlib.svh
│   └── tb_milestone2.sv
├── 02_test/
│   ├── isa_test/
│   └── c_program/
├── 03_sim/
│   └── makefile
└── 90_doc/
```

Directory purposes:

- **`00_src`**: Processor and other SystemVerilog design files.
- **`01_bench`**: Testbench and verification files.
- **`02_test`**: Test programs, binaries, and reference data.
- **`03_sim`**: Compilation and simulation scripts.
- **`90_doc`**: Student report and result documents for submission.

> ! **CRITICAL DO NOT MODIFY**
>
> - **`00_src/milestone2.sv`**
> - Supplied files under **`01_bench`**
> - Supplied programs or binaries under **`02_test`**
> - Existing commands or settings in **`03_sim/makefile`**
>
> Grading uses a clean copy of the course infrastructure, so changes to these files are ignored. If necessary, add separate files for custom tests or commands instead.

5.2. Milestone 2 Wrapper

The supplied `milestone2.sv` is the fixed wrapper between the student design and the course testbench.

The files `singlecycle.sv`, `imem.sv`, and `dmem.sv` are templates that define the interface of the three modules adapted to this wrapper.

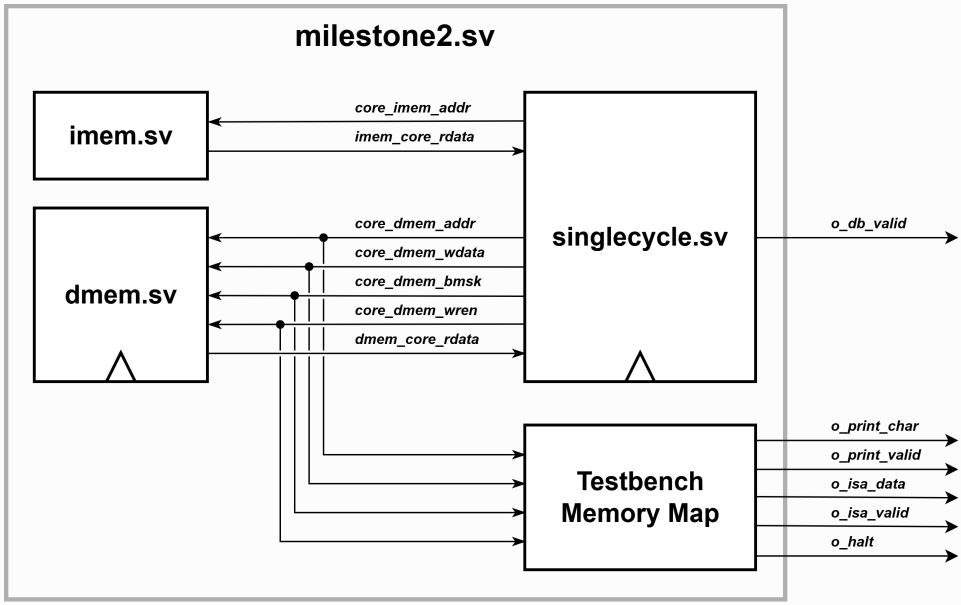


Figure 4: Milestone 2 wrapper and student-designed components.

Students must set the IMEM and DMEM start addresses and sizes according to the memory map below.

Table 10: Milestone 2 simulation memory map.

Region	C Programs	ISA Test	Description
Instruction Memory			
Reset vector	`0x8000_0000`	`0x8000_0000`	Initial program counter.
IMEM range	`0x8000_0000` – `0x8007_FFFF`	`0x8000_0000` – `0x83FF_FFFF`	Byte-addressed instruction memory.
IMEM size	512 KiB	64 MiB	Required memory capacity.
Address bits	19	26	Address offset width.
Data Memory			
DMEM range	`0x8008_0000` – `0x8008_FFFF`	`0x8400_0000` – `0x8400_FFFF`	Byte-addressed data memory.
DMEM size	64 KiB	64 KiB	Required memory capacity.
Address bits	16	16	Address offset width.

5.3. Simulation

The provided Makefile in `03_sim` supports compilation, simulation, and waveform viewing. Run all simulation commands from this directory:

{/} terminal

```
cd 03_sim
```

Before running simulations, log in to the compute node and load Cadence Xcelium:

{/} terminal

```
module load xcelium
```

The Makefile provides the following commands:

Table 11: Common Makefile commands.

Command	Description
`make help`	Show commands, options, and examples
`make filelist`	Generate the HDL file list (`flist`)
`make list_test`	List available tests and programs
`make sim`	Compile and run the selected test
`make wave`	Open the latest SHM waveform
`make clean`	Remove generated files, except `flist`

Run `make help` for detailed usage instructions, simulation options and exmaples.

First Run

Before the first simulation, generate the HDL file list:

{/} terminal

```
make filelist
```

This command scans through `00_src` and `01_bench` and generates `03_sim/flist`. Rerun this command whenever HDL or header files are added, removed, or renamed. Editing existing files does not require regeneration.

Simulation Options

Pass options to `make sim` using `OPTION=value`. Options can be combined in any order. Omitted options use their default values.

- `TEST=<name>`: Select the ISA test or C program. (Default: `ISA_TEST`)
- `IMEM_FORMAT=1B|4B`: Select the 8-bit or 32-bit IMEM format. (Default: `1B`)
- `WAVE=SHM|NONE`: Enable or disable waveform recording. (Default: `SHM`)
- `WAVE_START_TIME=<n>`: Waveform start time in ns. (Default: `0`)
- `WAVE_STOP_TIME=<n>`: Waveform stop time in ns. (Default: `TIMEOUT`)
- `MAX_CYCLES=<n>`: Maximum cycles before timeout. (Default: `5,000,000`)

Example commands:

{/} terminal

```
# Run ISA test with 8-bit IMEM
make sim TEST=ISA_TEST

# Run ISA test with 32-bit IMEM
make sim TEST=ISA_TEST IMEM_FORMAT=4B

# Run C program without waveform recording
make sim TEST=PROG_MATRIX WAVE=NONE

# Record waveforms between 8400000 ns and 8500000 ns
make sim TEST=ISA_TEST WAVE_START_TIME=8400000 WAVE_STOP_TIME=8500000

# Open recorded waveform
make wave
```

Notes:

- `IMEM_FORMAT` must match the IMEM implementation.
- Waveform settings do not affect simulation execution.
- Increase `MAX_CYCLES` only if a correct program requires more cycles.

Available Tests and Programs

The simulation environment provides two types of tests:

- **`ISA_TEST`**: Complete self-checking ISA test with focused behavioral groups.
- **`PROG_*`**: Precompiled C programs that exercise multiple RV32I instructions.

Use **`make list_test`** to display all available test names.

Table 12: Available tests and programs.

Value	Description
ISA Tests	
ISA_TEST	Runs the complete graded RV32I self-check
Precompiled C Programs	
PROG_HANOI	Solves and validates the 12-disk Tower of Hanoi
PROG_SORT	Compares six signed-integer sorting algorithms
PROG_MATRIX	Multiplies and cross-checks signed 8 by 8 matrices
PROG_SOFTFLOAT	Checks arithmetic across four floating-point formats
PROG_SIGMOID	Validates sigmoid behavior and three-class inference
PROG_CONVOLUTION	Applies Gaussian and Sobel kernels to a signed 7 by 7 matrix

Use the test name exactly as shown by **`make list_test`**. For example:

{/} terminal

```
make sim TEST=ISA_TEST
make sim TEST=PROG_MATRIX
```

Recommended Testing Order

It is recommended to verify the processor incrementally before running C programs.

1. Ensure the instruction fetch starts at the reset address specified in Table 10.
2. Run **`ISA_TEST`** and fix any reported failures.
3. Repeat **`ISA_TEST`** until all instruction groups pass.
4. Run at least three provided C programs to verify the complete processor.

If a C program fails or times out, return to **`ISA_TEST`** to identify and fix the underlying instruction errors.

Reading Test Results

The ISA test reports `PASS` or `FAIL` for each instruction and behavioral group. The final summary includes test results, cycle count, instruction count, IPC, and ISA score.

A successful simulation should report:

```text
============================================================
SIMULATION SUMMARY
============================================================
Total tests passed   : 37/37
Check groups passed  : 86/86
Total cases passed   : 404988/404988
Total cycles         : ...
Total instructions   : ...
IPC                  : ...
ISA score            : 6.00/6.00
============================================================
```

A `FAIL` indicates incorrect behavior in the corresponding group. Always debug the **first failure**, as later failures may be consequences of earlier errors.

Debugging with Waveforms

To avoid large waveform files, first run the ISA test without recording:

```terminal
make sim TEST=ISA_TEST WAVE=NONE
```

Identify the failure time from the test results, then record a short interval around it:

```terminal
make sim TEST=ISA_TEST WAVE_START_TIME=1000000 WAVE_STOP_TIME=1100000
make wave
```

The waveform is saved to `03_sim/wave.shm` and opened in SimVision using `make wave`.

6. Assessment

> **PROBLEM**
>
> Implement and verify a complete single-cycle RV32I processor using the provided interfaces. The processor must pass the ISA test and execute at least 3 supplied application programs.

- **Weight:** 30% of the overall laboratory grade.
- **Duration:** 2 weeks.

The lab TA will evaluate the submitted RTL using the original testbench, selected application programs, and a final Q&A session. All tests will run in the official grading environment.

6.1. Submission Requirements

Before submission:

- Run `ISA_TEST` and review the final score.
- Successfully execute at least 3 supplied programs. Testing all programs is recommended.
- Run `make clean` in `03_sim` to remove generated simulation files.
- Submit the source code and report following the prescribed procedure.
- Preserve the original project template.

Note: FPGA implementation is not required for this milestone.

6.2. Evaluation

- **6 points:** ISA test results based on passed behavioral groups.
- **2 points:** Successful execution of at least three supplied programs.
- **2 points:** Demonstration of design understanding in the final Q&A.